Index Terms: Computation and Language (cs.CL); Artificial Intelligence (cs.AI); Machine Learning (cs.LG)
Citations:9620

DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models

Zhihong Shao[1,2*†], Peiyi Wang[1,3*†], Qihao Zhu[1,3*†], Runxin Xu[1], Junxiao Song[1]
Xiao Bi[1], Haowei Zhang[1], Mingchuan Zhang[1], Y.K. Li[1], Y. Wu[1], Daya Guo[1*]
[1]DeepSeek-AI, [2]Tsinghua University, [3]Peking University

arXiv:2402.03300v3 [cs.CL] 27 Apr 2024
DeepSeek-AI
Code: https://github.com/deepseek-ai/DeepSeek-Math

1 Introduction

Research Problem

- **Mathematical reasoning:** requires structured, multi-step solutions rather than surface-level pattern matching.

- **Open-model gap:** strong systems such as GPT-4 and Gemini Ultra were not publicly available; open models trailed behind.

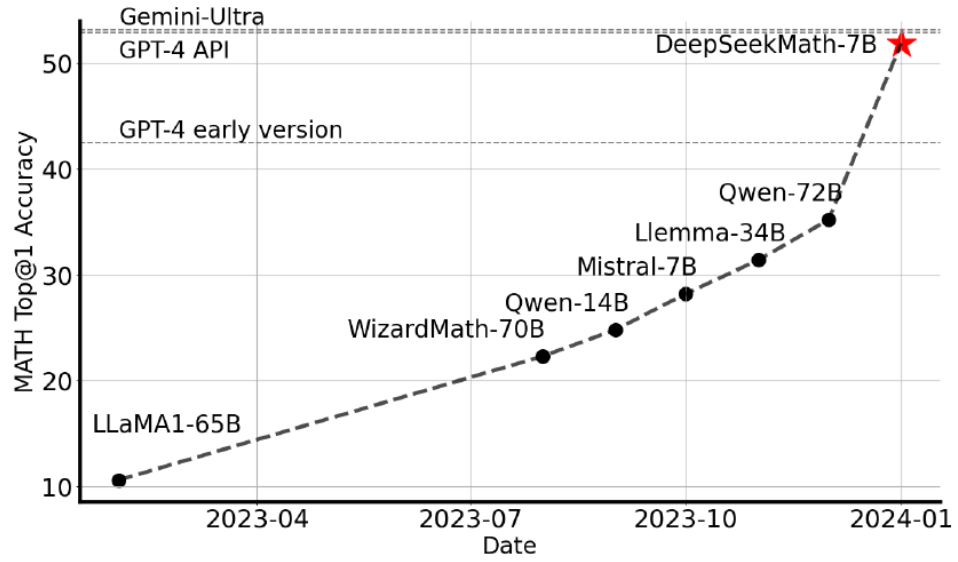


Figure 1 | Top1 accuracy of open-source models on the competition-level MATH benchmark (Hendrycks et al., 2021) without the use of external toolkits and voting techniques.

1.1 Contributions

Math Pre-Training at Scale

• **DeepSeekMathCorpus:** a high-quality dataset of 120B tokens from web pages filtered for mathematical Content; 9 times the size of the recently released OpenWebMath

• **Competitive small model:** DeepSeekMath-Base 7B reaches 36.2% on MATH, outperforming much larger prior systems such as Minerva 540B on this benchmark.

• **Code pre-training:** starting from a code-trained model improves mathematical reasoning.

• **No notable improvements with training on arXiv papers in this paper**

Exploration and Analysis of Reinforcement Learning

• **GRPO:** removes the separate value/critic model and uses relative scores of multiple outputs; reducing training resources compared to PPO

• **SFT-to-RL gains:** GSM8K improves 82.9% → 88.2%; MATH improves 46.8% → 51.7%.

• **Unified paradigm:** analyzes SFT, RFT, DPO, PPO and GRPO through data source, reward function and gradient coefficient.

1.2 Summary of Evaluations and Metrics

English and Chinese Mathematical Reasoning

GSM8K · MATH · SAT · OCW
MMLU-STEM · CMATH
Gaokao-MathCloze · Gaokao-MathQA

evaluate models' ability to generate self-contained text solutions without tool use, and also the ability to solve problems using Python.

Formal Mathematics:

evaluate DeepSeekMath-Base using the informal-to-formal theorem proving task from (Jiang et al., 2022) on miniF2F (Zheng et al., 2021) with Isabelle (Wenzel et al., 2008) chosen to be the proof assistant; strong few-shot autoformalization performance

Natural Language Understanding, Reasoning, and Code:

evaluate DeepSeekMath-Base on the Massive Multitask Language Understanding (MMLU) Benchmark, BIG-Bench Hard (BBH) for tasks that mostly require multi-step reasoning to solve ; HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021) which are widely used to evaluate code language models.

2 Math Pre-Training

1. **Initialize:** DeepSeek-Coder-Base-v1.5 7B.
2. **Build corpus:** 120B math tokens collected from Common Crawl.
3. **Continue pre-training:** 500B-token mixed training run.
4. **Instruction-tune:** 776K math instruction examples.
5. **Reinforcement learning:** GRPO produces DeepSeekMath-RL.

Design Rationale

- **Data:** filter a large public web crawl into a high-quality multilingual math corpus.
- **Initialization:** use DeepSeek-Coder-Base-v1.5 7B rather than a general-purpose base model.
- **Optimization:** combine continued pre-training, math-specific supervised fine-tuning and group-relative RL.

The 120B-token DeepSeekMath Corpus is the math-specific data source; the 500B tokens refer to the full Base-model continuation mix.

2 Math Pre-Training

***Math seed:** OpenWebMath, , a collection of high-quality mathematical web text

***Randomly select 500,000 data points from the seed corpus as positive training examples and another 500,000 web pages from Common Crawl as negative ones**

*use https://fasttext.cc employ an open-source library for training; configuring the vector dimension to 256, learning rate to 0.1, the maximum length of word n-gram to 3, the minimum number of word occurrences to 3, and the number oftraining epochs to 3; employ URL-based deduplication and near-deduplication techniques, resulting in 40B HTML web pages.

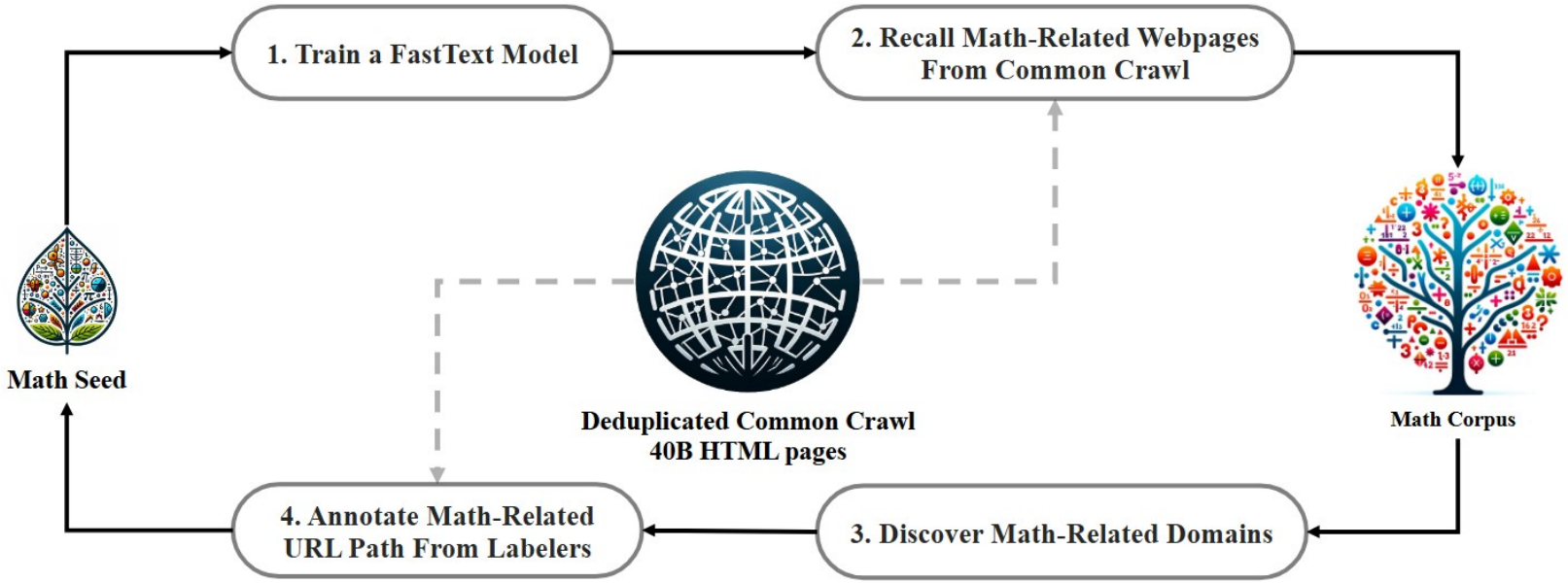


Figure 2 | An iterative pipeline that collects mathematical web pages from Common Crawl.

2.1 Data Collection and Decontamination

Iterative Common Crawl Mining Pipeline

1. **Seed corpus:** OpenWebMath supplies the positive examples.
2. **Train classifier:** fastText uses 500K positive and 500K negative examples.
3. **Recall pages:** retrieve and deduplicate about 40B HTML pages.
4. **Rank and filter:** retain the top 40B tokens in the first iteration.
5. **Improve the seed:** find mathematical domains and annotate URL paths.
6. **Repeat:** after four iterations, the corpus contains 35.5M pages and 120B tokens.

The fourth iteration adds little new content: nearly 98% of the final data had already been collected by iteration three.

2.1 Data Collection and Decontamination

Classifier configuration

256-dimensional vectors
Learning rate 0.1
Word n-gram = 3
Min word occurrences = 3
3 training epochs

Deduplication

URL-based deduplication
Near-duplicate removal
Reduced Common Crawl to
40B HTML pages

Iterative enrichment

Domains with >10% first-round collection flagged as math-related
Human annotators label relevant URL paths to expand the seed

Benchmark Decontamination

Main filtering rule

Remove a text segment if it contains an exact matching 10-gram
from an evaluation benchmark.

Short benchmark items

If a benchmark text is shorter than 10 grams but has ≥ 3 grams,
use exact matching to filter contaminated pages.

- **Benchmarks protected:** GSM8K, MATH, CMATH and AGIEval.
- **Purpose:** reduce evaluation leakage between pre-training pages and benchmark questions/answers.

2.2 Validating the Quality of the DeepSeekMath Corpus

Downstream results of DeepSeek-LLM 1.3B trained on different math corpora (few-shot CoT)

Math Corpus	Size	English Benchmarks					Chinese Benchmarks		
		GSM8K	MATH	OCW	SAT	MMLU STEM	CMATH	Gaokao MathCloze	Gaokao MathQA
No Math Training	N/A	2.9%	3.0%	2.9%	15.6%	19.5%	12.3%	0.8%	17.9%
MathPile	8.9B	2.7%	3.3%	2.2%	12.5%	15.7%	1.2%	0.0%	2.8%
OpenWebMath	13.6B	11.5%	8.9%	3.7%	31.3%	29.6%	16.8%	0.0%	14.2%
Proof-Pile-2	51.9B	14.3%	11.2%	3.7%	43.8%	29.2%	19.9%	5.1%	11.7%
DeepSeekMath Corpus	**120.2B**	**23.8%**	**13.6%**	**4.8%**	**56.3%**	**33.1%**	**41.5%**	**5.9%**	**23.6%**

Table 1 | Performance of DeepSeek-LLM 1.3B trained on different mathematical corpora, evaluated using few-shot chain-of-thought prompting. Corpus sizes are calculated using our tokenizer with a vocabulary size of 100K.

separately train a model on each mathematical corpus for 150B tokens; experiments are conducted using the efficient and light-weight HAI-LLM (High-flyer, 2023) training framework. Following the training practice of DeepSeek LLMs, we use the AdamW optimizer (Loshchilov and Hutter, 2017) with $\beta 1 = 0.9$, $\beta 2 = 0.95$, and weight_decay = 0.1,

2.2 Validating the Quality of the DeepSeekMath Corpus

8 BENCHMARKS

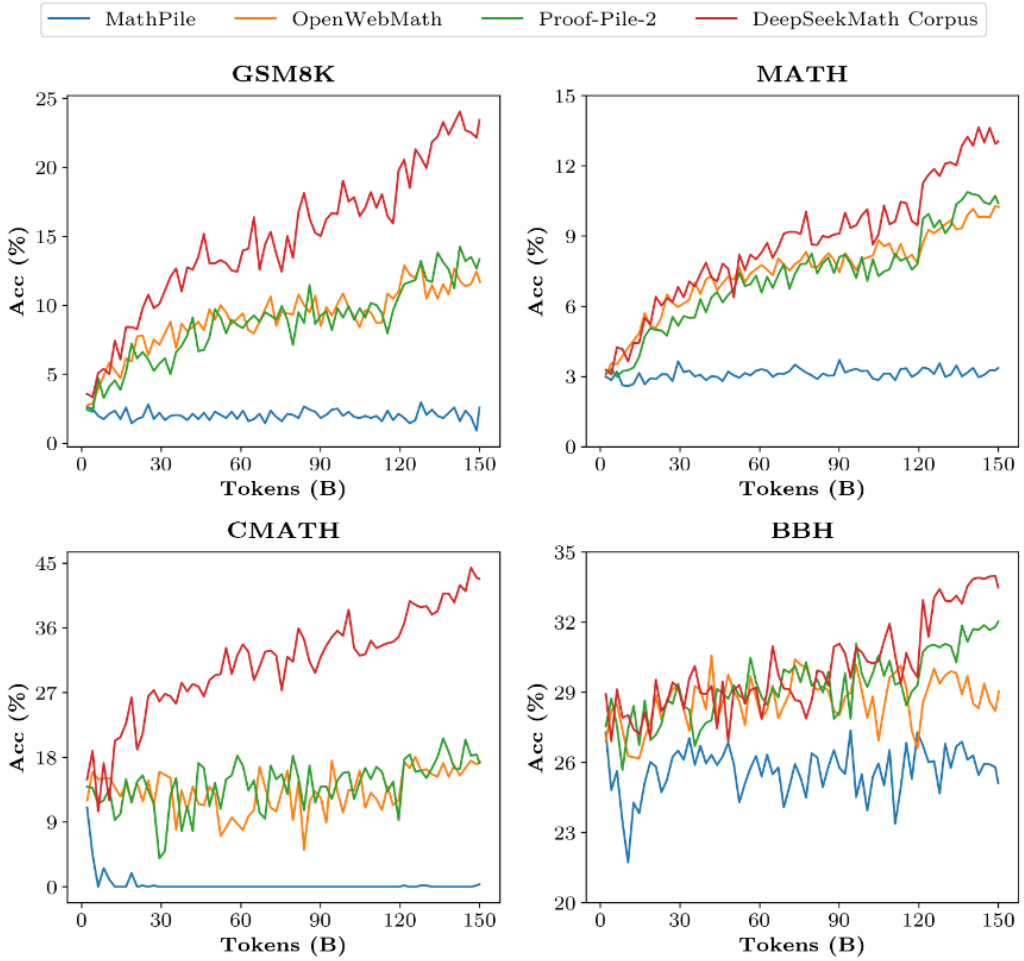


Figure 3 | Benchmark curves of DeepSeek-LLM 1.3B trained on different mathematical corpora.

2.3 Training and Evaluating DeepSeekMath-Base 7B

Continual Pre-Training Mixture · 500B Tokens



Training Configuration

Initialization	Token budget	Peak learning rate	Batch / context
DeepSeek-Coder-Base-v1.5 7B	500B total continual training	4.2×10^{-4}	10M tokens / 4K

• **Objective:** continued pre-training on the mixture preserves code capability while increasing mathematical and general reasoning.

2.3 Training and Evaluating DeepSeekMath-Base 7B

Model	Size	English Benchmarks					Chinese Benchmarks		
		GSM8K	MATH	OCW	SAT	MMLU STEM	CMATH	Gaokao MathCloze	Gaokao MathQA
Closed-Source Base Model									
Minerva	7B	16.2%	14.1%	7.7%	-	35.6%	-	-	-
Minerva	62B	52.4%	27.6%	12.0%	-	53.9%	-	-	-
Minerva	540B	58.8%	33.6%	17.6%	-	63.9%	-	-	-
Open-Source Base Model									
Mistral	7B	40.3%	14.3%	9.2%	71.9%	51.1%	44.9%	5.1%	23.4%
Llemma	7B	37.4%	18.1%	6.3%	59.4%	43.1%	43.4%	11.9%	23.6%
Llemma	34B	54.0%	25.3%	10.3%	71.9%	52.9%	56.1%	11.9%	26.2%
DeepSeekMath-Base	7B	**64.2%**	**36.2%**	**15.4%**	**84.4%**	**56.5%**	**71.7%**	**20.3%**	**35.3%**

Table 2 | Comparisons between DeepSeekMath-Base 7B and strong base models on English and Chinese mathematical benchmarks. Models are evaluated with chain-of-thought prompting. Minerva results are quoted from Lewkowycz et al. (2022a).

Model	Size	Problem Solving w/ Tools		Informal-to-Formal Proving	
		GSM8K+Python	MATH+Python	miniF2F-valid	miniF2F-test
Mistral	7B	48.5%	18.2%	18.9%	18.0%
CodeLlama	7B	27.1%	17.2%	16.3%	17.6%
CodeLlama	34B	52.7%	23.5%	18.5%	18.0%
Llemma	7B	41.0%	18.6%	20.6%	22.1%
Llemma	34B	64.6%	26.3%	21.0%	21.3%
DeepSeekMath-Base	7B	**66.9%**	**31.4%**	**25.8%**	**24.6%**

Table 3 | Few-shot evaluation of base models' ability to solve mathematical problems using tools and the ability to conduct informal-to-formal theorem proving in Isabelle.

Model	Size	MMLU	BBH	HumanEval (Pass@1)	MBPP (Pass@1)
Mistral	7B	**62.4%**	55.7%	28.0%	41.4%
DeepSeek-Coder-Base-v1.5†	7B	42.9%	42.9%	40.2%	52.6%
DeepSeek-Coder-Base-v1.5	7B	49.1%	55.2%	**43.2%**	**60.4%**
DeepSeekMath-Base	7B	54.9%	**59.5%**	40.9%	52.6%

Table 4 | Evaluation on natural language understanding, reasoning, and code benchmarks. DeepSeek-Coder-Base-v1.5† is the checkpoint right before learning rate decay, which is used to train DeepSeekMath-Base. On MMLU and BBH, we use few-shot chain-of-thought prompting. On HumanEval and MBPP, we evaluate model performance under the zero-shot setting and a few-shot setting, respectively.

3 Supervised Fine-Tuning

From Base Model to Instruction-Following Model

1. **Base model:** DeepSeekMath-Base 7B after continued pre-training.
2. **SFT dataset:** 776K curated mathematical instruction examples.
3. **Fine-tuning:** supervised fine-tuning with a 4K context length.
4. **Resulting model:** DeepSeekMath-Instruct 7B, supporting CoT, PoT and tool use.

Three Solution Formats

Chain-of-Thought (CoT)	Program-of-Thought (PoT)	Tool-Integrated Reasoning
Natural-language, step-by-step derivation	Generate code to carry out computation	Combine reasoning with external Python execution

SFT turns the strong base model into a model explicitly trained to respond to mathematical instructions.

3.1 SFT Data Curation

776K	English	Chinese	Solution diversity
Total instruction examples	GSM8K, MATH, MathInstruct, Lila-OOD subsets	K–12 questions across 76 sub-topics	CoT, PoT and tool-integrated solutions

Dataset Construction

- **English:** GSM8K and MATH are annotated with tool-integrated solutions; a subset of MathInstruct and Lila-OOD supplies CoT/PoT-style reasoning.
- **Mathematics coverage:** algebra, probability, number theory, calculus and geometry.
- **Chinese:** K–12 math problems include both CoT and tool-integrated solution formats, covering 76 sub-topics such as linear equations.
- **Training packaging:** examples are randomly concatenated to a maximum context length of 4K tokens.

3.2 Training and Evaluating DeepSeekMath-Instruct 7B

Instruction-Tuning Setup

Base checkpoint	Training steps	Batch size	Learning rate
DeepSeekMath-Base 7B	500	256	5×10^{-5} (constant)

Evaluation Protocol

- **Benchmarks:** GSM8K and MATH (English), MGSM-zh and CMATH (Chinese).
- **No-tool setting:** generate self-contained solutions using step-by-step reasoning.
- **Tool-use setting:** allow Python execution / program-based reasoning.
- **Sampling:** paper reports top-1 and majority-vote results; majority voting uses 32 candidates in the reported tables.

3.2 Training and Evaluating DeepSeekMath-Instruct 7B

Interpretation

• **MATH:** DeepSeekMath-Instruct reaches 57.4% when Python-assisted reasoning is allowed.

• **Relative scale:** it is competitive with much larger open models, though tool-integrated results are protocol-dependent.

• **Role of tools:** program execution can handle arithmetic reliably, complementing the model's natural-language plan.

4 Reinforcement Learning

RL Fine-Tuning Pipeline

1. **Prompts:** start from GSM8K and MATH chain-of-thought questions.
2. **Sample group:** draw G candidate outputs from the old policy.
3. **Score:** assign each output a reward.
4. **Compute advantage:** normalize rewards within the group.
5. **Update policy:** optimize the GRPO objective with KL regularization.

Training Focus

- **Starting model:** DeepSeekMath-Instruct 7B.
- **RL data:** chain-of-thought-format questions related to GSM8K and MATH, selected from the SFT data (~144K questions).
- **Research purpose:** test whether reinforcement learning improves the already strong instruction-tuned model, including out-of-domain benchmarks.
- **Core design:** sample several answers to the same question and compare their rewards, instead of training a separate critic model.

The selected RL data is intentionally narrower than the complete SFT dataset; this makes the generalization effect measurable.

4.1 Group Relative Policy Optimization

PPO: Actor–Critic

- **PPO components:** question q, policy $\pi\theta$, reward model $r\varphi$, and a separate value model $V\psi$.

GRPO: Group-Relative Setup

- **GRPO setup:** sample G outputs for each question and normalize their reward scores to obtain $\hat{A}$.
- **Key difference:** GRPO does not require a separate value/critic model.

- **Memory saving:** PPO often needs an extra value model comparable in size to the policy.
- **Relative baseline:** GRPO compares rewards among outputs for the same question.
- **Regularization:** KL divergence keeps the trained policy closer to the reference model.

4.1.1 From PPO to GRPO

Equation (1) · PPO Clipped Objective

Equation (1)

$$\mathcal{J}_{PPO}(\theta) = \mathbb{E}\left[q \sim P(Q), o \sim \pi_{\theta_{old}}(O|q)\right] \frac{1}{|o|} \sum_{t=1}^{|o|} \min\left[\frac{\pi_\theta(o_t|q,o_{<t})}{\pi_{\theta_{old}}(o_t|q,o_{<t})} A_t, \text{clip}\left(\frac{\pi_\theta(o_t|q,o_{<t})}{\pi_{\theta_{old}}(o_t|q,o_{<t})}, 1-\varepsilon, 1+\varepsilon\right) A_t\right], \quad (1)$$

• **A_t:** advantage estimated with Generalized Advantage Estimation (GAE) and a learned value function $V\psi$..

• **ε:** clipping hyperparameter that limits excessively large policy updates.

$\pi\theta$ and $\pi\theta old$ are the current and old policy models

q,o are questions and outputs

~ means Distributed as

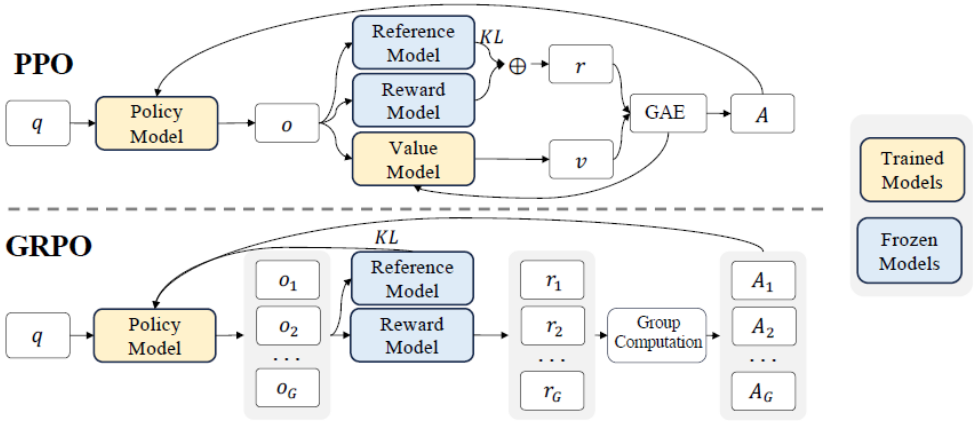


Figure 4 | Demonstration of PPO and our GRPO. GRPO foregoes the value model, instead estimating the baseline from group scores, significantly reducing training resources.

The PPO formulation uses a learned value function; GRPO replaces this baseline with group-relative rewards.

4.1.1 From PPO to GRPO

Equation (2) · Per-token KL penalty in PPO reward

$$r_t = r_\varphi(q, o_{\leq t}) - \beta \log \frac{\pi_\theta(o_t|q,o_{<t})}{\pi_{ref}(o_t|q,o_{<t})}, \quad (2)$$

$r\varphi$ is the reward model, πref is the reference model, which is usually the initial SFT model, and β is the coefficient of the KL penalty.

The PPO formulation uses a learned value function; GRPO replaces this baseline with group-relative rewards.

4.1.1 From PPO to GRPO

Equation (3) · GRPO Objective

Equation (3)

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}\left[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(O|q)\right]$$
$$\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\left\{\min\left[\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q,o_{i,<t})}\hat{A}_{i,t}, \text{clip}\left(\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q,o_{i,<t})}, 1-\varepsilon, 1+\varepsilon\right)\hat{A}_{i,t}\right] - \beta\mathbb{D}_{KL}\left[\pi_\theta \| \pi_{ref}\right]\right\}, \quad (3)$$

What Changes from PPO?

Sample a group	**Group-relative advantage**	**No critic model**
G outputs for the same question q	$\hat{A}_{i,t}$ depends on reward relative to siblings	Less auxiliary model memory and computation

ε and β are hyper-parameters, and $\hat{A}_{i,t}$ is the advantage calculated based on relative rewards of the outputs inside each group only, which will be detailed in the following subsections.

$\rho_{i,t}$ is the policy ratio for token t of sampled output i. Exact objective is written compactly for readability.

4.1.1 From PPO to GRPO

Equation (4) · Unbiased KL-Divergence Estimator

Equation (4)

$$\mathbb{D}_{KL}\left[\pi_\theta \parallel \pi_{ref}\right] = \frac{\pi_{ref}\left(o_{i,t}|q, o_{i,<t}\right)}{\pi_\theta\left(o_{i,t}|q, o_{i,<t}\right)} - \log\frac{\pi_{ref}\left(o_{i,t}|q, o_{i,<t}\right)}{\pi_\theta\left(o_{i,t}|q, o_{i,<t}\right)} - 1, \qquad (4)$$

Role of KL Regularization

- **Reference policy:** usually the initial SFT model, π_ref.
- **In PPO:** the KL term is incorporated into the reward at each token.
- **In GRPO:** the KL divergence is added directly to the optimization objective, keeping the group-relative advantage calculation separate.
- **Estimator:** the paper uses this positive unbiased estimator (Schulman, 2020).

4.1.2

4.1.3

Group-Level Reward Normalization

Outcome Supervision RL with GRPO

Process Supervision RL with GRPO

Â_i,t = r̃_i for every token t in output o_i

$$\hat{A}_{i,t} = \tilde{r}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})},$$

Procedure

$$\tilde{r}_i^{index(j)} = \frac{r_i^{index(j)} - \text{mean}(\mathbf{R})}{\text{std}(\mathbf{R})}.$$

- **1. Sample:** for a question q, draw G candidate outputs from the old policy.
- **2. Score:** the reward model assigns one scalar reward to each completed output.
- **3. Normalize:** subtract the group mean and divide by the group standard deviation.
- **4. Broadcast:** use the normalized result as the advantage for all tokens in that output, then maximize Equation (3).

$$\hat{A}_{i,t} = \sum_{index(j) \geq t} \tilde{r}_i^{index(j)}$$

Outcome supervision provides feedback at the end of each output rather than at each reasoning step.

4.1.3 Process Supervision RL with GRPO

Step-Level Reward Normalization

Process-supervision advantages

$$\tilde{r}_{i,\mathrm{index}(j)} = [\, r_{i,\mathrm{index}(j)} - \mathrm{mean}(R) \,] \,/\, \mathrm{std}(R)$$

$$\hat{A}_{i,t} = \Sigma_{\{\mathrm{index}(j) \geq t\}}\, \tilde{r}_{i,\mathrm{index}(j)}$$

Where Rewards Enter the Reasoning Chain

Step 1	Step 2	Step 3	Final
reward r_i^1	reward r_i^2	reward r_i^3	reward r_i^K

Difference from Outcome Supervision

- **Outcome supervision:** one reward at the end; every token in that answer receives the same normalized advantage.
- **Process supervision:** a process reward model scores each reasoning step; each token receives the sum of normalized rewards from the current and following steps.
- **Motivation:** fine-grained feedback may be more informative for complex multi-step mathematics.
- **Optimization:** both variants maximize the same GRPO objective in Equation (3); the advantage construction differs.

4.1.4

Algorithm 1 Iterative Group Relative Policy Optimization

Input initial policy model $\pi_{\theta_{\text{init}}}$; reward models r_φ; task prompts $\mathcal{D}$; hyperparameters ε, β, μ

policy model $\pi_\theta \leftarrow \pi_{\theta_{\text{init}}}$
for iteration = 1, . . . , I **do**
    reference model $\pi_{ref} \leftarrow \pi_\theta$
    for step = 1, . . . , M **do**
        Sample a batch $\mathcal{D}_b$ from $\mathcal{D}$
        Update the old policy model $\pi_{\theta_{old}} \leftarrow \pi_\theta$
        Sample G outputs $\{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(\cdot \mid q)$ for each question $q \in \mathcal{D}_b$
        Compute rewards $\{r_i\}_{i=1}^{G}$ for each sampled output o_i by running r_φ
        Compute $\hat{A}_{i,t}$ for the t-th token of o_i through group relative advantage estimation.
        for GRPO iteration = 1, . . . , μ **do**
            Update the policy model π_θ by maximizing the GRPO objective (Equation 21)
    Update r_φ through continuous training using a replay mechanism.

Output π_θ

4.1.4 Iterative RL with GRPO

Algorithm 1 · Iterative Group Relative Policy Optimization

1. **Initialize policy:** $\pi\theta \leftarrow \pi\theta_init$.
2. **Set reference:** $\pi_ref \leftarrow$ current $\pi\theta$.
3. **Sample outputs:** draw G outputs per prompt.
4. **Score and normalize:** convert rewards into group advantages.
5. **Update policy:** maximize Equation (3).
6. **Refresh reward model:** use new samples together with replay data.

- **Replay mechanism:** retain 10% of historical reward-model data during continuous reward-model training.
- **Iteration benefit:** update the reward model to supervise the current policy as it changes.
- **Paper experiment:** two RL iterations were tested, with a substantial gain in the first iteration.

4.2 Training and Evaluating DeepSeekMath-RL

Reinforcement-Learning Training Setup

Policy initialization	RL training data	Reward model LR	Policy LR
DeepSeekMath-Instruct 7B	~144K CoT questions from GSM8K + MATH	2×10^{-5}	1×10^{-6}

GRPO Hyperparameters

KL coefficient	Samples / question	Maximum length	Batch size
$\beta = 0.04$	G = 64 outputs	1,024 tokens	1,024

- **Data restriction:** only CoT-format SFT questions from GSM8K and MATH are used in RL; other SFT questions are excluded.
- **Evaluation:** GSM8K and MATH are treated as in-domain; other mathematical benchmarks measure transfer to out-of-domain tasks.

Hyperparameters transcribed from Section 4.2 of the paper.

4.2 Training and Evaluating DeepSeekMath-RL

Model	Size	English Benchmarks		Chinese Benchmarks	
		GSM8K	MATH	MGSM-zh	CMATH
Chain-of-Thought Reasoning					
Closed-Source Model					
Gemini Ultra	-	94.4%	53.2%	-	-
GPT-4	-	92.0%	52.9%	-	86.0%
Inflection-2	-	81.4%	34.8%	-	-
GPT-3.5	-	80.8%	34.1%	-	73.8%
Gemini Pro	-	86.5%	32.6%	-	-
Grok-1	-	62.9%	23.9%	-	-
Baichuan-3	-	88.2%	49.2%	-	-
GLM-4	-	87.6%	47.9%	-	-
Open-Source Model					
InternLM2-Math	20B	82.6%	37.7%	-	-
Qwen	72B	78.9%	35.2%	-	-
Math-Shepherd-Mistral	7B	84.1%	33.0%	-	-
WizardMath-v1.1	7B	83.2%	33.0%	-	-
DeepSeek-LLM-Chat	67B	84.1%	32.6%	74.0%	80.3%
MetaMath	70B	82.3%	26.6%	66.4%	70.9%
SeaLLM-v2	7B	78.2%	27.5%	64.8%	-
ChatGLM3	6B	72.3%	25.7%	-	-
WizardMath-v1.0	70B	81.6%	22.7%	64.8%	65.4%
DeepSeekMath-Instruct	7B	82.9%	46.8%	73.2%	84.6%
DeepSeekMath-RL	7B	**88.2%**	**51.7%**	**79.6%**	**88.8%**
Tool-Integrated Reasoning					
Closed-Source Model					
GPT-4 Code Interpreter	-	97.0%	69.7%	-	-
Open-Source Model					
InternLM2-Math	20B	80.7%	54.3%	-	-
DeepSeek-LLM-Chat	67B	86.7%	51.1%	76.4%	85.4%
ToRA	34B	80.7%	50.8%	41.2%	53.4%
MAmmoTH	70B	76.9%	41.8%	-	-
DeepSeekMath-Instruct	7B	83.7%	57.4%	72.0%	84.3%
DeepSeekMath-RL	7B	**86.7%**	**58.8%**	**78.4%**	**87.6%**

Table 5 | Performance of Open- and Closed-Source models with both Chain-of-Thought and Tool-Integrated Reasoning on English and Chinese Benchmarks. Scores in gray denote majority votes with 32 candidates; The others are Top1 scores. DeepSeekMath-RL 7B beats all open-source models from 7B to 70B, as well as the majority of closed-source models. Although DeepSeekMath-RL 7B is only further trained on chain-of-thought-format instruction tuning data of GSM8K and MATH, it improves over DeepSeekMath-Instruct 7B on all benchmarks.

Main Finding

- **Broad improvement:** RL trained on GSM8K/MATH CoT data improves all four benchmarks shown.
- **Beyond training data:** Chinese benchmarks and tool-integrated tasks also improve even though they are not all directly supervised in RL.
- **Paper claim:** DeepSeekMath-RL 7B exceeds 50% top-1 accuracy on competition-level MATH without external tools or voting.

5 Discussion

Discussion Structure

5.1 Lessons Learnt in Pre-Training	5.2 Insights of Reinforcement Learning	6 Conclusion
Code training; arXiv data; corpus design	Unified view of RL methods; why RL helps	Data scale + SFT + GRPO

Questions the Discussion Addresses

- **Pre-training:** does code training help math? Do arXiv papers provide useful math-training signal?
- **Reinforcement learning:** is the gain caused by better fundamental capability, more robust output distributions, or both?
- **Unified analysis:** how do the data source, reward function and gradient coefficient differ across SFT, RFT, DPO, PPO and GRPO?
- **Future direction:** how can data, algorithms and reward models make RL more effective?

5.1 Lessons Learnt in Pre-Training

Code and Math Training Recipes · DeepSeek-LLM 1.3B

Training Setting	Training Tokens			w/o Tool Use			w/ Tool Use	
	General	Code	Math	GSM8K	MATH	CMATH	GSM8K+Python	MATH+Python
No Continual Training	–	–	–	2.9%	3.0%	12.3%	2.7%	2.3%
Two-Stage Training								
Stage 1: General Training	400B	–	–	2.9%	3.2%	14.8%	3.3%	2.3%
Stage 2: Math Training	–	–	150B	19.1%	14.4%	37.2%	14.3%	6.7%
Stage 1: Code Training	–	400B	–	5.9%	3.6%	19.9%	12.4%	10.0%
Stage 2: Math Training	–	–	150B	**21.9%**	**15.3%**	**39.7%**	17.4%	9.4%
One-Stage Training								
Math Training	–	–	150B	20.5%	13.1%	37.6%	11.4%	6.5%
Code & Math Mixed Training	–	400B	150B	17.6%	12.1%	36.3%	**19.7%**	**13.5%**

Table 6 | Investigation of how code affects mathematical reasoning under different training settings. We experiment with DeepSeek-LLM 1.3B, and evaluate its mathematical reasoning performance without and with tool use via few-shot chain-of-thought prompting and few-shot program-of-thought prompting, respectively.

training settings outlined in Section 2.2.1.
use an 89B-token dataset from
the second iteration of
the data collection process

Takeaways

- **Code first, then math:** best no-tool mathematical scores among the listed training recipes.
- **Mixed code + math:** performs best for the shown Python-assisted scores.
- **Training order matters:** token count alone does not explain performance.

5.1.1 Code Training Benefits Mathematical Reasoning

General Reasoning and Coding · Table 7

Training Setting	Training Tokens			MMLU	BBH	HumanEval (Pass@1)	MBPP (Pass@1)
	General	Code	Math				
No Continual Training	–	–	–	24.5%	28.1%	12.2%	13.0%
Two-Stage Training							
Stage 1: General Training	400B	–	–	25.9%	27.7%	15.2%	13.6%
Stage 2: Math Training	–	–	150B	33.1%	32.7%	12.8%	13.2%
Stage 1: Code Training	–	400B	–	25.0%	31.5%	25.0%	**40.0%**
Stage 2: Math Training	–	–	150B	**36.2%**	35.3%	12.2%	17.0%
One-Stage Training							
Math Training	–	–	150B	32.3%	32.5%	11.6%	13.2%
Code & Math Mixed Training	–	400B	150B	33.5%	**35.6%**	**29.3%**	39.4%

Table 7 | Investigation of how different settings of code and math training affect model performance of language understanding, reasoning, and coding. We experiment with DeepSeek-LLM 1.3B. We evaluate the models on MMLU and BBH using few-shot chain-of-thought prompting. On HumanEval and MBPP, we conduct zero-shot and few-shot evaluations, respectively.

Model	Size	ArXiv Corpus	English Benchmarks					Chinese Benchmarks		
			GSM8K	MATH	OCW	SAT	MMLU STEM	CMATH	Gaokao MathCloze	Gaokao MathQA
DeepSeek-LLM	1.3B	No Math Training	2.9%	3.0%	2.9%	15.6%	19.5%	12.3%	0.8%	17.9%
		MathPile	2.7%	3.3%	2.2%	12.5%	15.7%	1.2%	0.0%	2.8%
		ArXiv-RedPajama	3.3%	3.4%	4.0%	9.4%	9.0%	7.4%	0.8%	2.3%
DeepSeek-Coder-Base-v1.5	7B	No Math Training	29.0%	12.5%	6.6%	40.6%	38.1%	45.9%	5.9%	21.1%
		MathPile	23.6%	11.5%	7.0%	46.9%	35.8%	37.9%	4.2%	25.6%
		ArXiv-RedPajama	28.1%	11.1%	7.7%	50.0%	35.2%	42.6%	7.6%	24.8%

Table 8 | Effect of math training on different arXiv datasets. Model performance is evaluated with few-shot chain-of-thought prompting.

Interpretation

- **Coding capability:** code + math mixing improves HumanEval and MBPP.
- **Broader effect:** math training can also benefit general reasoning and language tasks.

5.1.2 ArXiv Papers Seem Ineffective in Improving Mathematical Reasoning

Formal-Proving Evaluation · miniF2F (Isabelle)

ArXiv Corpus	miniF2F-valid	miniF2F-test
No Math Training	20.1%	21.7%
MathPile	16.8%	16.4%
ArXiv-RedPajama	14.8%	11.9%

Table 9 | Effect of math training on different arXiv corpora, the base model being DeepSeek-Coder-Base-v1.5 7B. We evaluate informal-to-formal proving in Isabelle.

Data sources tested

MathPile: 8.9B tokens, >85% arXiv

ArXiv-RedPajama: 28B tokens

Full arXiv LaTeX after cleaning

Authors' Interpretation

- **No consistent gain:** arXiv-only corpora show no notable improvement and can reduce mathematical-reasoning scores.
- **Possible nuance:** the paper notes that future work should test arXiv when mixed with other data, for informal-to-formal theorem proving, and at larger model scales.
- **Do not overgeneralize:** the result concerns the corpora and experimental settings tested here, not a claim that every arXiv document is useless.

5.2.1 Towards a Unified Paradigm

Equation (5) · General Gradient Form

Equation (5)

$$\nabla_\theta \mathcal{J}_{\mathcal{A}}(\theta) = \mathbb{E}\left[\underbrace{(q,o) \sim \mathcal{D}}_{\text{Data Source}}\right]\left(\frac{1}{|o|}\sum_{t=1}^{|o|} \underbrace{GC_{\mathcal{A}}(q,o,t,\pi_{rf})}_{\text{Gradient Coefficient}} \nabla_\theta \log \pi_\theta\,(o_t|q,o_{<t})\right), \quad (5)$$

Three Ingredients

D · Data source

Determines prompts and sampled outputs

r / π_ref · Reward source

Rule-based judgment or a learned reward model

A · Algorithm

Produces GC, the token-level gradient coefficient

GC controls how strongly each token probability is reinforced or penalized.

The appendix derives specific choices of data source and GC for SFT, RFT, Online RFT, DPO, PPO and GRPO.

5.2 Insights of Reinforcement Learning

Methods	Data Source	Reward Function	Gradient Coefficient
SFT	$q, o \sim P_{sft}(Q, O)$	-	1
RFT	$q \sim P_{sft}(Q), o \sim \pi_{sft}(O\|q)$	Rule	Equation 10
DPO	$q \sim P_{sft}(Q), o^+, o^- \sim \pi_{sft}(O\|q)$	Rule	Equation 14
Online RFT	$q \sim P_{sft}(Q), o \sim \pi_\theta(O\|q)$	Rule	Equation 10
PPO	$q \sim P_{sft}(Q), o \sim \pi_\theta(O\|q)$	Model	Equation 18
GRPO	$q \sim P_{sft}(Q), \{o_i\}_{i=1}^{G} \sim \pi_\theta(O\|q)$	Model	Equation 21

Table 10 | The data source and gradient coefficient of different methods. P_{sft} denotes the data distribution of supervised fine-tuning datasets. $\pi_{\theta_{sft}}$ and π_θ denote the supervised fine-tuned model and the real-time policy model during the online training process, respectively.

Three Components in the Paper's Analysis

- **Data source:** prompts and outputs determine which examples train the policy.
- **Reward + algorithm:** scores become token-level weights that reinforce or penalize outputs.

5.2 Insights of Reinforcement Learning

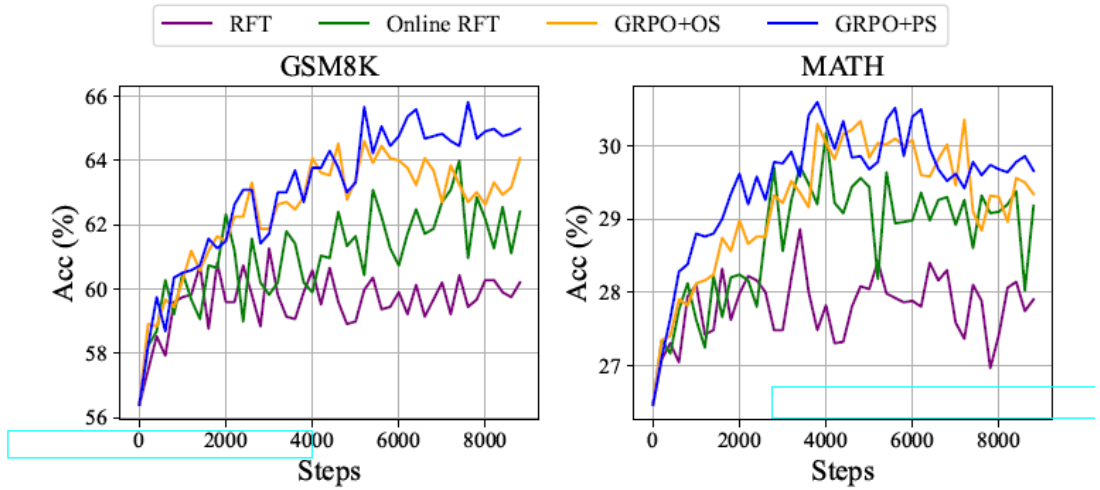


Figure 5 | Performance of the DeepSeekMath-Instruct 1.3B model, which was further train using various methods, on two benchmarks.

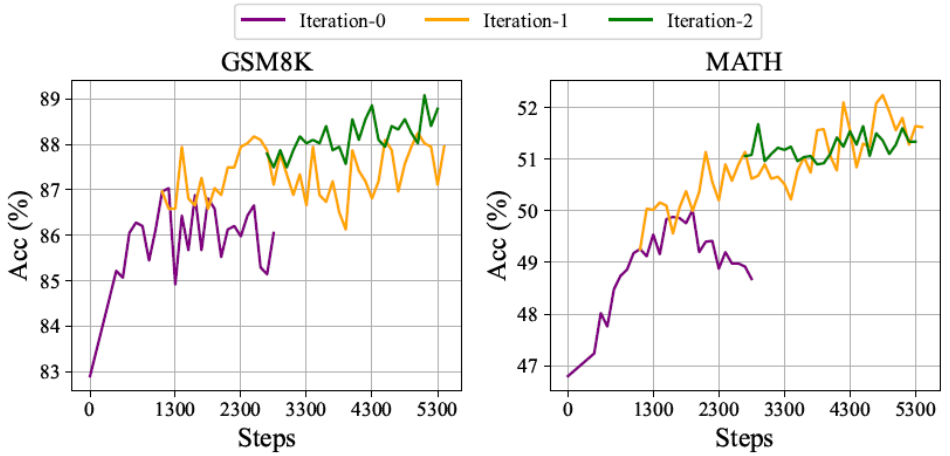


Figure 6 | Performance of iterative reinforcement learning with DeepSeekMath-Instruct 7B on two benchmarks.

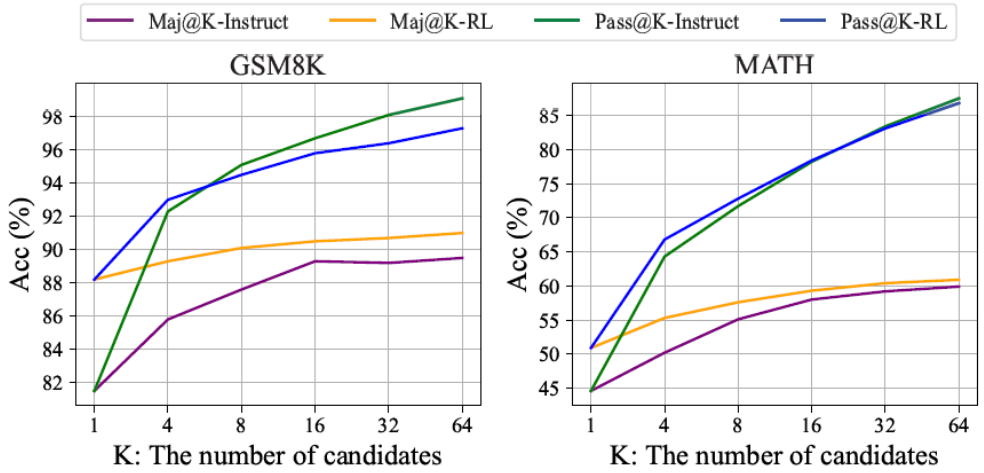


Figure 7 | The Maj@K and Pass@K of SFT and RL DeepSeekMath 7B on GSM8K and MATH (temperature 0.7). It was noted that RL enhances Maj@K but not Pass@K.

A.1.1 Supervised Fine-Tuning

Equations (6)–(7) · SFT Objective and Gradient

Equation (6)

$$J_{SFT}(\theta) = \mathbb{E}_{[q,o\sim P_{sft}(Q,O)]}\left(\frac{1}{|o|}\sum_{t=1}^{|o|}\log \pi_\theta\left(o_t|q,o_{<t}\right)\right).$$

Equation (7)

$$\nabla_\theta J_{SFT} = \mathbb{E}_{[q,o\sim P_{sft}(Q,O)]}\left(\frac{1}{|o|}\sum_{t=1}^{|o|}\nabla_\theta \log \pi_\theta\left(o_t|q,o_{<t}\right)\right).$$

Interpretation

- **Data source:** question–answer pairs selected for supervised fine-tuning.
- **Reward function:** implicit human selection of the target response; no separate reward model is required.
- **Gradient coefficient:** always 1 for target tokens; standard next-token log-likelihood learning.

A.1.2 Rejection Sampling Fine-tuning

Equations (8)–(10) · RFT Objective, Gradient and Coefficient

Equation (8)

$$J_{RFT}(\theta) = \mathbb{E}_{[q\sim P_{sft}(Q),o\sim\pi_{sft}(O|q)]}\left(\frac{1}{|o|}\sum_{t=1}^{|o|}\mathbb{I}\,(o)\log\pi_\theta\,(o_t|q,o_{<t})\right).$$

Equation (9)

$$\nabla_\theta J_{RFT}(\theta) = \mathbb{E}_{[q\sim P_{sft}(Q),o\sim\pi_{sft}(O|q)]}\left(\frac{1}{|o|}\sum_{t=1}^{|o|}\mathbb{I}\,(o)\nabla_\theta\log\pi_\theta\,(o_t|q,o_{<t})\right).$$

Equation (10)

$$GC_{RFT}(q,o,t) = \mathbb{I}(o) = \begin{cases}1 & \text{the answer of o is correct}\\ 0 & \text{the answer of o is incorrect}\end{cases}$$

- **Procedure:** sample multiple outputs from the SFT model, keep correct outputs, then fine-tune on accepted outputs.
- **Implication:** correct answers receive positive learning weight; incorrect answers receive zero weight in this simplified formulation.

A.1.3 Online Rejection Sampling Fine-tuning

Equation (11) · Online RFT Gradient

Equation (11)

$$\nabla_\theta J_{OnRFT}(\theta) = \mathbb{E}_{[q\sim P_{sft}(Q),o\sim\pi_\theta(O|q)]}\left(\frac{1}{|o|}\sum_{t=1}^{|o|}\mathbb{I}\,(o)\nabla_\theta \log \pi_\theta\,(o_t|q,o_{<t})\right).$$

Difference from Offline RFT

Offline RFT	Online RFT
Outputs are sampled from the fixed SFT model π_SFT.	Outputs are sampled from the current policy π_θ.

- **Shared rule signal:** I(o) indicates answer correctness.
- **Main distinction:** the training examples are refreshed from the real-time policy, allowing exploration to track current behavior.

A.1.4 Direct Preference Optimization

Equation (12) · DPO Objective

Equation (12) · Conditional context is omitted from token notation

$$J_{DPO}(\theta) = \mathbb{E}_{[q\sim P_{sft}(Q),o^+,o^-\sim\pi_{sft}(O|q)]} \log\sigma\left(\beta\frac{1}{|o^+|}\sum_{t=1}^{|o^+|}\log\frac{\pi_\theta(o_t^+|q,o_{<t}^+)}{\pi_{ref}(o_t^+|q,o_{<t}^+)} - \beta\frac{1}{|o^-|}\sum_{t=1}^{|o^-|}\log\frac{\pi_\theta(o_{<t}^-|q,o_{<t}^-)}{\pi_{ref}(o_{<t}^-|q,o_{<t}^-)}\right)$$

Equation (13) · Pairwise Gradient

Equation (13)

$$\nabla_\theta J_{DPO}(\theta) = \mathbb{E}_{[q\sim P_{sft}(Q),o^+,o^-\sim\pi_{sft}(O|q)]}\left(\frac{1}{|o^+|}\sum_{t=1}^{|o^+|} G\, C_{DPO}(q,o,t)\nabla_\theta\log\pi_\theta\,(o_t^+|q,o_{<t}^+) - \frac{1}{|o^-|}\sum_{t=1}^{|o^-|} G\, C_{DPO}(q,o,t)\nabla_\theta\log\pi_\theta\,(o_t^-|q,o_{<t}^-)\right)$$

Equation (14) · Token-level coefficient (abbreviated)

$$GC_{DPO}(q,o,t) = \sigma\left(\beta\log\frac{\pi_\theta(o_t^-|q,o_{<t}^-)}{\pi_{ref}(o_t^-|q,o_{<t}^-)} - \beta\log\frac{\pi_\theta(o_t^+|q,o_{<t}^+)}{\pi_{ref}(o_t^+|q,o_{<t}^+)}\right)$$

o^+ is the preferred output, o^- the rejected output; full formulas condition each token probability on q and preceding tokens.

A.1.5 Proximal Policy Optimization

Equation (15) · PPO Objective

Equation (15)

$$J_{PPO}(\theta) = \mathbb{E}_{\left[q\sim P_{sft}(Q), o\sim\pi_{\theta_{old}}(O|q)\right]} \frac{1}{|o|}\sum_{t=1}^{|o|} \min\left[\frac{\pi_\theta(o_t|q,o_{<t})}{\pi_{\theta_{old}}(o_t|q,o_{<t})} A_t, \text{clip}\left(\frac{\pi_\theta(o_t|q,o_{<t})}{\pi_{\theta_{old}}(o_t|q,o_{<t})}, 1-\varepsilon, 1+\varepsilon\right) A_t\right].$$

Equations (16)–(17) · Simplified Objective and Gradient

Equation (16)

$$J_{PPO}(\theta) = \mathbb{E}_{\left[q\sim P_{sft}(Q), o\sim\pi_{\theta_{old}}(O|q)\right]} \frac{1}{|o|}\sum_{t=1}^{|o|} \frac{\pi_\theta(o_t|q,o_{<t})}{\pi_{\theta_{old}}(o_t|q,o_{<t})} A_t.$$

Equation (17)

$$\nabla_\theta J_{PPO}(\theta) = \mathbb{E}_{\left[q\sim P_{sft}(Q), o\sim\pi_{\theta_{old}}(O|q)\right]} \frac{1}{|o|}\sum_{t=1}^{|o|} A_t \nabla_\theta \log \pi_\theta\left(o_t|q,o_{<t}\right)$$

Equation (18)

$$GC_{PPO}\left(q, o, t, \pi_{\theta_{rm}}\right) = A_t,$$

A_t is computed with GAE from token rewards and a learned value function Vψ; PPO therefore needs a critic/value model.

A.1.6 Group Relative Policy Optimization

Equation (19) · GRPO Objective (Simplified Analysis)

Equation (19) · Token conditioning on q and previous tokens is abbreviated

$$J_{GRPO}(\theta) = \mathbb{E}_{\left[q\sim P_{sft}(Q),\{o_i\}_{i=1}^{G}\sim\pi_{\theta_{old}}(O|q)\right]}\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\left[\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q,o_{i,<t})}\hat{A}_{i,t} - \beta\left(\frac{\pi_{ref}(o_{i,t}|q,o_{i,<t})}{\pi_\theta(o_{i,t}|q,o_{i,<t})} - \log\frac{\pi_{ref}(o_{i,t}|q,o_{i,<t})}{\pi_\theta(o_{i,t}|q,o_{i,<t})} - 1\right)\right].$$

Equation (20) · GRPO Gradient

Equation (20)

$$\nabla_\theta J_{GRPO}(\theta) = \mathbb{E}_{\left[q\sim P_{sft}(Q),\{o_i\}_{i=1}^{G}\sim\pi_{\theta_{old}}(O|q)\right]}\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\left[\hat{A}_{i,t} + \beta\left(\frac{\pi_{ref}(o_{i,t}|o_{i,<t})}{\pi_\theta(o_{i,t}|o_{i,<t})} - 1\right)\right]\nabla_\theta \log\pi_\theta\left(o_{i,t}|q,o_{i,<t}\right).$$

Equation (21)

$$GC_{GRPO}(q,o,t,\pi_{\theta_{rm}}) = \hat{A}_{i,t} + \beta\left(\frac{\pi_{ref}(o_{i,t}|o_{i,<t})}{\pi_\theta(o_{i,t}|o_{i,<t})} - 1\right)$$

This simplified equation assumes π_old = π_θ for the appendix derivation. The main training objective uses the clipped form in Equation (3).

5.2.2 Why RL Works?

Pass@K vs. Maj@K

Pass@K	**Maj@K**
Does at least one of K samples solve the problem?	Does the majority / most frequent answer become correct?

Observed Effect

- **RL improves Maj@K:** the model becomes more likely to produce a correct answer consistently across samples.
- **Pass@K does not improve similarly:** the best-of-K capability is not the metric showing the main RL gain.
- **Authors' interpretation:** RL may primarily make the output distribution more robust by increasing the probability of correct responses among sampled candidates, rather than creating wholly new underlying capabilities.
- **Implication:** reasoning evaluation should distinguish "can produce a correct answer at least once" from "usually produces a correct answer."

5.2.3 How to Achieve More Effective RL?

Data Source

- **Broader prompts:** improve generalization to out-of-distribution problems and advanced decoding or tree-search strategies.
- **Efficient exploration:** inference-time efficiency directly affects how much policy exploration can be done.

Algorithm

- **Robustness to noise:** the algorithm should avoid blindly trusting unreliable reward signals.
- **Weak-to-strong alignment:** use algorithmic improvements to reduce dependence on perfect supervision.

Reward Function

- **OOD generalization:** reward models must score new questions and advanced outputs reliably.
- **Uncertainty:** represent reward-model uncertainty, potentially to avoid over-optimizing noisy scores.
- **Combine signals:** rule-based correctness and learned reward models offer complementary supervision.

Potential Limitation Highlighted by the Paper

Even carefully annotated process-reward data (PRM800K) is reported to contain approximately 20% incorrect annotations, illustrating how noisy reward signals can become a bottleneck.

These are research directions discussed by the authors, not completed methods demonstrated by this paper.

6 Conclusion

Main Contributions

Scalable math corpus

120B math tokens selected from Common Crawl

Math-trained 7B model

Code-trained initialization + 500B-token mix

Memory-efficient RL

GRPO removes the separate value model

Headline Results

88.2%

GSM8K · RL · CoT top-1

51.7%

MATH · RL · CoT top-1

60.9%

MATH · 64-sample self-consistency

7B

Model parameter scale

The paper's central message: carefully selected mathematical data, code-pretrained initialization, mathematical SFT, and group-relative policy optimization together produce strong open-model mathematical reasoning.

References

Primary Source

Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y. K., Wu, Y., & Guo, D. (2024). DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300v3.

Benchmarks and Methods Cited Within the Paper

- **Math benchmarks:** GSM8K, MATH, CMATH, MGSM-zh, SAT, OCW, MMLU-STEM, Gaokao-MathQA and miniF2F.
- **Training methods:** SFT, Rejection Sampling Fine-tuning (RFT), Direct Preference Optimization (DPO), PPO and GRPO.
- **Evaluation caveat:** scores depend on prompting, tool access, sampling and voting settings reported in the paper.